

Module 3 : Divide and Conquer & Greedy Method.

→ Divide and Conquer: -

- In algo design, the problem we want to solve can be divided into two or more subproblems.

- Smaller in size, easier to solve the ultimately an lead to final solution.

- To divide and conquer approach, we solve a problem recursively applying the 3 steps recursively at each level of the recursion.

- (1) Divide: The problem into a number of subproblems that are smaller instances of the problem.

- (2) Conquer: The subproblem by solving them recursively.

- (3) Combine: The solution of subproblems into the solution for original problem.

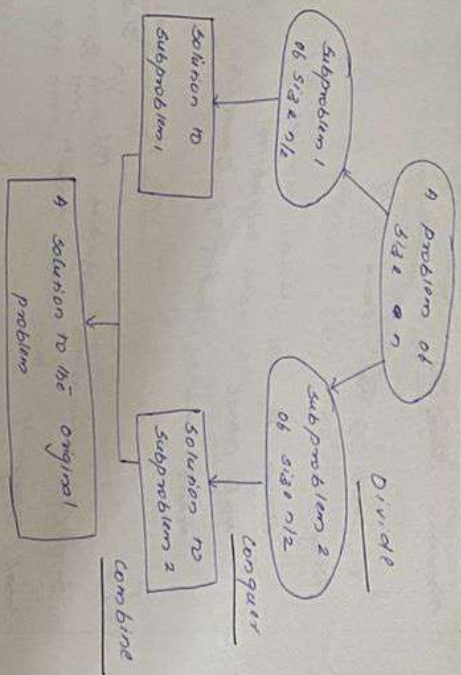
- Recursive algorithm have two cases:

(1) Recursion case

(2) Base case

- Recurrence is used to represent the running time of recursive algorithm.

- The recurrence of divide & conquer depends on three steps of basic program



- E.g: Binary search

merge sort

quicksort

→ Control abstraction of Divide and Conquer: -

- A control abstraction is a procedure

- whose block of control is clear

- whose primary operations are specified by other procedures.

Other procedures.

- whose precise meaning are left undefined.

- The control abstraction for divide and conquer technique is DnDC(P), where P is the problem to be solved

else {
 divide P into smaller instances P1, P2, ..., Pk
 Apply Dnnc to each subproblem
 return combine(Dnnc(P1), Dnnc(P2), ..., Dnnc(Pk))
}

- Small(n) determines whether the 'if' side is small enough so that the answer can be computed without splitting.

- If this so function 'S' is inverted (solution).

- Otherwise, the problem P is divided into smaller subproblems.

These subproblems P1, P2, ..., Pk are solved by recursive application of Dnnc.

- combine is a function that determines the solution P using the solutions to the k subproblems.

→ Complexity analysis of Dnnc:—

- If the size of problem P is n and size of k subproblems are n1, n2, ..., nk then the computing time

of Dnnc is described by the recurrence relation

$$T(n) = \begin{cases} g(n) & n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{otherwise} \end{cases}$$

Time for solving problem P, n is size of P

f(n) → time for dividing P and combining the solutions for subproblems

T(n) → Time for Dnnc on any 'if' of size n.

g(n) → Time to compute the answer directly for small inputs.

5, 8, 10, 12, 21, 7, 9

- the complexity of merge divide and conquer algo is given by recurrence of the form:

$$T(n) = \begin{cases} T(n) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

a, b → constant

T(n) is known as n is a power of b (ie, n = b^k)

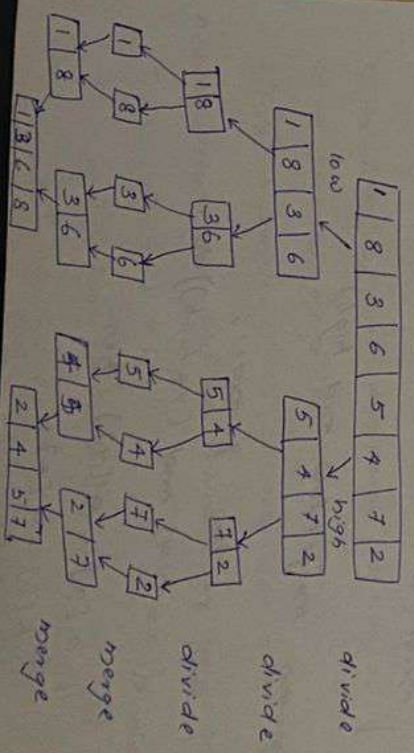
→ 2-way merge sort:-

- Given a sequence of n elements a[1] ... a[n]. Split this array into 2 sets: a[1] ... a[n/2] and a[n/2+1] ... a[n].

- Each set is individually sorted and the resulting sorted sequences are merged to produce a single sorted sequence of elements.

- To sort

1. Divide: Divide a element sequence to be sorted into 2 subsequences of n/2 element each
 2. Conquer: Sort the subsequences recursively
 3. Combine: merge the 2 sorted subsequences to produce the sorted sequence



divide
 divide
 merge
 merge

$$= 2^3 \cdot 7(n/2^3) + 3cn$$

$$= 2^4 \cdot 7(n/2^4) + 4cn$$

$$= 2^{\log n} \cdot 7(1) + \log n \cdot cn$$

$$= 7cn + \log n \cdot cn$$

$$= 2cn + cn \log n$$

$$= O(n \log n)$$

Best, average and worst case complexity of merge sort = $O(n \log n)$

→ Strassen's Matrix Multiplication:—

- We can multiply 2 matrices A and B if the no. of columns of A = no. of rows of B.

- Consider a square matrix of size n (n is power of 2). If n is not power of 2, we can make it to be adding zeros.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}_{2 \times 2} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}_{2 \times 2} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}_{2 \times 2}$$

Where $C = A \times B$, The matrix C can be calculated as,

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

$$c_{22} = a_{21}b_{12} + a_{22}b_{22}$$

- A normal matrix multiplication on square matrix of order n will take $O(n^3)$ time.

- $\Theta(n^3) \rightarrow$ Access each element of matrix.

- $\Theta(n^3) \rightarrow$ multiply n^2 elements.

- By using this method we can reduce the complexity of matrix multiplication to $\Theta(2.81)$

- Strassen's method reduces the time to 4 steps:

(i) Divide the input matrix A and B to 4 sub-matrices. It will take $\Theta(1)$ time.

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

(ii) Create 10 matrices $S_1, S_2, \dots, S_{10}$ each of which is $n/2 \times n/2$ and is the sum of 2 matrices created in step (i) will take $\Theta(n^2)$ time.

$$S_1 = B_{12} - B_{22}$$

$$S_2 = A_{11} + A_{12}$$

$$S_3 = A_{21} + A_{22}$$

$$S_4 = B_{21} + B_{11}$$

$$S_5 = A_{11} + A_{22}$$

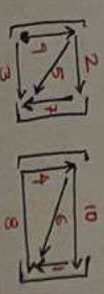
$$S_6 = B_{11} + B_{22}$$

$$S_7 = A_{12} - A_{22}$$

$$S_8 = B_{21} - B_{22}$$

$$S_9 = A_{21} - A_{22}$$

$$S_{10} = B_{11} + B_{12}$$



$$C_4 = P + S - T + V = 6 + 6 - 6 + 0 = 6$$

$$C_{12} = P + T = (-1) + 6 = 5$$

$$C_{21} = Q + S = 0 + 6 = 6$$

$$C_{22} = P + R - Q + U = 6 + (-1) - 0 + (-1) = 4$$

$$AB = \begin{bmatrix} 6 & 5 \\ 6 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & 1 \\ 2 & 0 & 0 & 2 \\ 0 & 1 & 2 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 0 & 0 & 2 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\Rightarrow P = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$A_{11} + A_{22} = \begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 3 & 1 \end{bmatrix}$$

$$B_{11} + B_{22} = \begin{bmatrix} 2 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 4 & 2 \\ 10 & 4 \end{bmatrix}$$

$$S = A_{22}(B_{21} - B_{11})$$

$$= \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} -1 & 1 \\ -1 & 0 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ -2 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ -2 & 2 \end{bmatrix}$$

$$Q = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 6 & 0 \end{bmatrix}$$

$$T = \begin{bmatrix} 1 & 1 \\ 3 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix}$$

$$U = \begin{bmatrix} 0 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 \\ 2 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ 0 & -2 \end{bmatrix}$$

$$V = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$C_{11} = \begin{bmatrix} 4 & 2 \\ 10 & 4 \end{bmatrix} + \begin{bmatrix} -1 & 0 \\ -2 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 \\ 5 & 1 \end{bmatrix}$$

$$C_{12} = \begin{bmatrix} -1 & 1 \\ -2 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix} = \begin{bmatrix} 0 & 3 \\ 1 & 7 \end{bmatrix}$$

$$C_{21} = \begin{bmatrix} 3 & 0 \\ 6 & 0 \end{bmatrix} + \begin{bmatrix} -1 & 0 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 4 & 1 \end{bmatrix}$$

$$C_{22} = \begin{bmatrix} 4 & 2 \\ 10 & 4 \end{bmatrix} + \begin{bmatrix} -1 & 1 \\ -2 & 2 \end{bmatrix} - \begin{bmatrix} 3 & 0 \\ 6 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 3 \\ 2 & 4 \end{bmatrix}$$

$$AB = \begin{bmatrix} 2 & 0 & 0 & 3 \\ 5 & 1 & 1 & 7 \\ 2 & 0 & 0 & 3 \\ 4 & 1 & 2 & 4 \end{bmatrix}$$

- Greedy algorithm are a problem solving technique which is suitable for optimisation problems.

- When divide and conquer and dynamic programming tries to find the best solution for the problem, greedy approach finds a best possible choice at that point.

i.e., we are not thinking about the past or future instance.

- Greedy take the best one at that point of iteration.

- Similar to DnD & DP greedy algorithm find the (greedy) optimal solution by dividing the problem in subproblems.

- We are finding the local optimal solution of subproblem, which leads to the global solution.

- There may be a more optimal solution (find by some of the previous methods) but the difference with greedy approach will be negligible.

- Two feature (features) of greedy algorithms are,

1. Greedy choice property: 'globally optimal

solution can be arrived at by making a locally optimal solution'. This local optimal solution is otherwise called 'greedy choice'.

2. Optimal structure: optimal solutions contain optimal subsolutions.

- Used to solve optimisation problems.

- Greedy algorithm seems by taking a sequence of choices or decisions to solve given problem.

- It takes the choice that seems best at the particular moment.

- Decisions are taken by some selection criteria which will hopefully given an optimal solution.

- Doesn't always leads to an optimal solution.

→ Control abstraction: —

Algorithm Greedy (0,0)

{

 If $a[1:n]$ contains 0 inputs

 solution = ϕ // initialise the solution as empty

 for $i = 1$ to n do

 {

$x = \text{select}(a_i)$;

 If feasible (solution, x) then

 solution. Union (solution, {x})

 }

}

 return solution

- array $a[1 \dots n]$ stores the inputs.

- solution set is initialised as null.

- with the help of some selection criteria on input x is chosen from the array.

- The function select selects an input from 'a'

whose value assign to x.

Steps:-

- 1) Compute the profit/weight ratio for each item.
- 2) Sort all the items in decreasing order of their profit/weight ratio.
- 3) Start filling the knapsack by putting the items into it one by one.

Algorithm:-

Algorithm knapsack(m, n)

// $P[i:n]$ and $w[i:n]$ contain the profits and weights respectively.

// n is the n -objects ordered such that $P[i]/w[i] \geq P[i+1]/w[i+1]$, n is the size of knapsack.

// $x[1:n]$ is the solution vector.

{
 for $i=1$ to n do $x[i] = 0.0$

$v := m$;

 for $i=1$ to n do

 {
 if $(w[i] > v)$ then break

$x[i] = 1.0$

 }
 $v := v - w[i]$

}
if $(i = n)$
 $x[i] = v/w[i]$
}

no. of objects
→
knapsack weight

Step 1: Find Profit/weight for each objects

2: Since we have to select whether we have to completely select the object or partially select it. So we sort the profit/weight is selected first.

3: The object with the highest profit/weight is selected first.

4: Mark the object with 1 if it is completely selected or the fraction part if it is not selected completely.

5: While we select a particular object, Deduct the knapsack size by its particular object size.

6: Repeat 4 & 5.

7: Note the final fractional part & count that objects in the knapsack.

8: Find the total weight and total profit earned.

Example:-

For the given set of items and knapsack capacity = 60 kg, find the optimal solution for the fractional knapsack problem.

$n = 5$, $w = 60 \text{ kg}$

$\{w_1, w_2, w_3, w_4, w_5\} = \{5, 10, 15, 22, 25\}$
 $\{P_1, P_2, P_3, P_4, P_5\} = \{30, 40, 45, 77, 90\}$

• Properties of spanning tree:-

- A connected graph G can have more than one spanning tree
- spanning tree is minimally connected.
- Doesn't have loop/cycle.
- Adding one edge makes loop i.e. spanning tree is minimally acyclic.
- spanning tree has $(n-1)$ edges, $n \rightarrow$ no. of vertices.

minimum no. of spanning trees of a complete graph with n nodes is:

$$\boxed{n^{n-2}}$$

Minimum spanning tree (MST):-

- A minimum spanning tree is a spanning tree that has minimum weight than all other trees of the same graph.

- Minimum spanning tree algorithms:

① Prim's algorithm

② Kruskal's algorithm

Applications of spanning tree:-

- * Civil network planning
- * Computer network routing protocol
- * Cluster analysis
- * Image segmentation
- * Handwriting recognition

Kruskal's algorithm:-

- Builds the spanning tree by adding edges one by one into a growing spanning tree.
- In each iteration "breaks" an edge which has least weight and add to the growing spanning tree.
- Edges of the graph are considered in the increasing order of its cost.
- If the selected edge will form a cycle, then discard it.
- The selection process continues until there are $n-1$ edges.

Algorithm: $\text{kruskal}(E, \text{cost}, n)$

{ Construct a heap out of edge costs using $\text{Heapify}(E)$;

for $i = 1$ to n do

$\text{parent}[i] = -1$;

$i = 0$

$\text{mincost} = 0$

 while $(i < n-1)$ and (heap not empty) do

 {

 Delete a minimum cost edge (u,v) from the

 heap and reheapify using $\text{Adjust}(i)$;

$j = \text{Find}(u)$;

$k = \text{Find}(v)$;

 if $j \neq k$ then

 {

$i = i + 1$

$t[u, v] = u$

$t[v, u] = v$;

$\text{mincost} = \text{mincost} + \text{cost}[u, v]$;

$\text{Union}(j, k)$;

 }

 }

if $n-1$ then
 return ("no spanning tree");

else
 return mincost

}

$E \rightarrow$ set of edges

$n \rightarrow$ no. of vertices.

$t \rightarrow$ set of edges in the minimum cost spanning tree

The final cost is returned.

- Heapify() $\rightarrow$ construct a minheap based on the edge cost of G.

- Adjust() $\rightarrow$ reconstruct a minheap if there is a deletion occurs.

- Find() $\rightarrow$ returns the set number of that particular vertex j and k are the set number of vertex u & v respectively

- If $j \neq k$ vertex u & v belongs to two different sets. Inclusion of (u, v) should definitely form cycle. So discard it

- If $j = k$ means vertex u & v belongs to different set. Inclusion of (u, v) should not form cycle. So add to MST

- Finally $n-1$ edges then return it.

Complexity: -

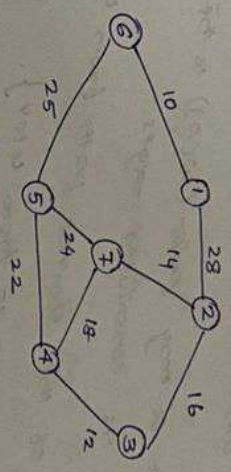
- The edges are maintained as a minheap, then the next edge to consider can be obtained in $O(\log V)$ times.

- Construction of heap itself takes $O(V \log V)$ times

- Overall complexity $O(E \log V)$.

Example: -

Construct the MST for the given graph using Kruskal's algorithm.



Step 1: (6,1) 10

Step 2: (3,4) 12

Step 3: (2,7) 14

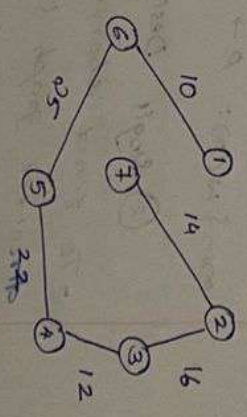
Step 4: (2,3) 16

Step 5: (7,4) 18 form cycle reject it.

Step 6: (5,4) 22

Step 7: (7,5) 24 reject.

Step 8: (6,5) 25



Min Cost = $10 + 25 + 22 + 12 + 16 + 14 = 99$

⇒ Compute P_i/w_i

	P_i	w_i	P_i/w_i
1	30	5	6
2	40	10	4
3	45	15	3
4	22	22	3.5
5	25	25	3.6

② Sort all the items decreasing order

	P_i	w_i	P_i/w_i
1	30	5	6
2	40	10	4
3	45	25	3.6
4	22	22	3.5
5	25	15	3

③ Start filling the knapsack by putting the items into it one by one

Item	P_i	w_i	Remaining weight	x_i	$P_i x_i$
1-1	30	5	60 - 5 = 55	1	1 x 30 = 30
2	40	10	55 - 10 = 45	1	1 x 40 = 40
3	45	25	45 - 25 = 20	1	1 x 45 = 45
4	22	22	Fraction 22/22	22/22	(22/22) x 22 = 22
5	25	15	Capacity limit - This is not included	0	0

$$\sum P_i x_i = 30 + 40 + 45 + 22 = 137$$

Solution vector $x[i]$ based on items $[i-1 \text{ to } i-5]$

$$(x_1, x_2, x_3, x_4, x_5) = (1, 1, 0, 20/22, 1)$$

$$(w_1, w_2, w_3, w_4, w_5) = (5, 10, 15, 22, 25)$$

$$(x_1, x_2, x_3, x_4, x_5) = (1, 1, 0, 20/22, 1)$$

$$\sum w_i x_i = 1 \times 5 + 1 \times 10 + 0 \times 15 + (20/22) \times 22 + 1 \times 25$$

$$= 5 + 10 + 20 + 25$$

$$= 60$$

$$\sum P_i x_i = 137$$

$$\text{Constraints: } \sum w_i x_i \leq m \quad 60 \leq 60$$

$$\text{Objective: maximize } \sum P_i x_i$$

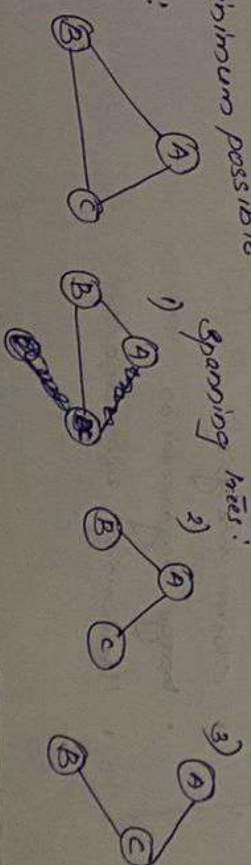
True

Minimum Cost Spanning Tree:-

* Spanning Tree:-

- Subset of undirected connected component graph $G=(V, E)$, which has all the vertices covered with minimum possible number of edges.

G_1 :



- Feasible is a boolean valued function that checks whether adding x to the solution set will still keep the solution feasible. If yes the input x will be added to the solution set.

- The function union combines x with the solution and update the objective function.

- The function greedy describes the essential way that a greedy algorithm will work, once a particular problem is chosen & the function select, feasible, & union are properly implemented.

→ Knapsack Pbm.

→ Problem Scenario: -

- A thief is robbing a store and can carry a maximal weight into his knapsack.

- There are n items available in the store and each with a weight and a profit.

- What items should be thief take.

- We need to put these items in a knapsack according to its capacity to get maximum profit.

- The items should be selected in such a way that the thief will carry those items for which he will gain maximum profit.

- Objective of the thief is to maximize the profit.

* Knapsack problems are categorised as:

- Fractional knapsack problem

- Items are divisible.

- Solved by greedy method.

- 0-1 knapsack problem

- Items can be indivisible.

- Solved by dynamic programming.

→ Fractional Knapsack Problem / Greedy Knapsack

Problem: -

- Given a knapsack of capacity m and n items to be considered to fill the knapsack with.

- The item i is associated with a weight w_i and a profit p_i .

- If a fraction x_i ($0 \leq x_i \leq 1$) of item i is placed into the knapsack then a profit of $p_i x_i$ is earned.

- Since the total capacity is m , the total weight of chosen items should be $\leq m$.

- The objective of knapsack problem is to maximize the profit within the capacity limit, subject to the following conditions.

$$\sum_{i=1}^n p_i x_i \Rightarrow \sum_{i=1}^n w_i x_i \leq m$$

- Greedy algorithm gives the optimal solution for the fractional knapsack problem.

Step (iii) compute 7 matrix products $P_1, P_2, \dots, P_7$
each matrix P_i is of $n/2 \times n/2$

$$P_1 = A_{11} S_1 = A_{11} B_{12} - A_{11} B_{22}$$

$$P_2 = S_2 B_{22} = A_{11} B_{22} + A_{12} B_{22}$$

$$P_3 = S_3 B_{11} = A_{21} B_{11} + A_{22} B_{11}$$

$$P_4 = S_4 A_{22} = B_{21} A_{22} - B_{11} A_{22}$$

$$P_5 = S_5 S_6 = (A_{11} + A_{22}) (B_{11} + B_{22})$$

$$P_6 = S_7 S_8 = (A_{12} - A_{22}) (B_{21} + B_{22})$$

$$P_7 = S_9 S_{10} = (A_{21} - A_{22}) (B_{11} + B_{12})$$

(iv) Compute various sub matrix of $C_{11}, C_{12}, C_{21}, C_{22}$ by adding and subtracting various combination of P_i this can be done by $\Theta(n^2)$ time

$$C_{11} = P_5 + P_4 - P_3 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_7 - P_3 + P_7$$

* Analysis:-

- Strassen algo. uses 7 multiplications which reduces the time complexity $T(n/2)$
- multiplication take more time than compared to addition & subtraction but the no. of addition and subtraction increased.
- Addition and subtraction of 2 matrices take $\Theta(n^2)$ time.

$$\therefore T(n) = \begin{cases} 1 & n \leq 2 \\ 7T(n/2) + \Theta(n^2) & n > 2 \end{cases}$$

By master's theorem

$$n \log_b^a = n \log_2^7$$

$$f(n) = O(n^{\log_b^a - \epsilon})$$

$$T(n) = \Theta(n^{\log_b^a})$$

$$= \Theta(n^{\log_2^7}) = \Theta(n^{2.81})$$

* Example:-

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix} \quad B = \begin{bmatrix} 0 & 1 \\ 3 & 2 \end{bmatrix} \quad \text{Find } AB$$

$$P = (A_{11} + A_{22}) (B_{11} + B_{22}) = (1+2)(0+2) = 6$$

$$Q = (A_{21} + A_{22}) B_{11} = (0+2)0 = 0$$

$$R = A_{11} (B_{12} - B_{22}) = 1(1-2) = -1$$

$$S = A_{22} (B_{21} - B_{11}) = 2(3-0) = 6$$

$$T = B_{22} (A_{11} + A_{12}) = 2(1+2) = 6$$

$$U = (A_{21} - A_{11}) (B_{11} + B_{12}) = (0-1)(0+1) = -1$$

$$V = (A_{12} - A_{22}) (B_{21} + B_{22}) = (2-2)(3+2) = 0$$

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

1. Divide the array into two parts
2. Divide the array into two parts again
3. Break each element into single parts
4. Sort the elements from smallest to largest
5. Merge the divided sorted array together
6. The array has been sorted.

* Algorithm mergesort:—

* Algorithm mergesort (low, high)

{
 // a (low, high) is a global array to be sorted.
 // Small (P) is true if there is only one element
 to sort. In this case the list is already sorted.

if (low < high) then // if there are more than 1 element
 // divide P into subproblems

 mid = (low + high) / 2 // finds where to split to set

 mergesort (low, mid) // sort one subset

 mergesort (mid + 1, high) // sort the other subset

 merge (low, mid, high) // combine the solutions.

{

* Algorithm merge (low, mid, high)

{
 i = low, x = low, y = mid + 1;

 while ((x ≤ mid) and (y ≤ high)) do

 {
 if a[x] ≤ a[y] then

 { b[i] = a[x];

 x = x + 1;

else
 { b[i] = a[y];

 i = i + 1;
 y = y + 1;

if (x ≤ mid) then

{
 for k = x to mid do

 { b[i] = a[k];

 i = i + 1

else

{
 for k = y to high do

 { b[i] = a[k];

 i = i + 1;

for k = low to high do

 a[k] = b[k]

* Time Complexity:—

$$T(n) = \begin{cases} a & n=1 \\ 2T(n/2) + cn & \text{otherwise} \end{cases}$$

a → time to sort an array of size 1

cn → time to merge two sub-arrays

2T(n/2) → complexity of 2 recursion calls

Above recurrence can be solved by iteration method

$$T(n) = 2T(n/2) + cn$$

$$= 2(2T(n/4) + c(n/2)) + cn$$

$$= 2^2 T(n/2^2) + 2cn$$

* Dividing or finding middle element $\Theta(1)$

* combine $\Theta(n)$

* Size of subarray 1/2 cost required to solve $T(n/2)$ total $2T(n/2)$

* single element in array $\Theta(1)$

→ Single Source Shortest Path Algorithms:-

- The shortest path problem is the problem of finding a path between two vertices in a graph such that the sum of the weights of its constituent edges is minimised.

- Different shortest path problems:

① Single source shortest path problem:

- Give a connected weighted graph $G=(V,E)$, find the shortest path from a given source vertex s to every other vertices ($v \in V$) in the graph.

- The weight of any path (wCPP) is the sum of the weights of its constituent edges.

- The weight of the shortest path from u to v = $\min\{wCPP: p \rightarrow \text{path from } u \text{ to } v\}$.

② Single Destination Shortest Path Problem:-

- To find shortest paths from all vertices in the directed graph to a single destination vertex v .

③ All pair shortest path problem: To find shortest paths between every pair of vertices in the graph.

Single source shortest path algorithms are:

- ① Bellman Ford Algorithm
- ② Dijkstra's Algorithm.

* Dijkstra's Algorithm:-

- Given a graph and a source vertex s in graph, find shortest path from s to all vertices in the given graph.

Algorithm Dijkstra(G, u, s)

1. For each vertex v in G
 - 1.1 $distance[v] = \infty$
 - 1.2 $previous[v] = NULL$
2. $distance[s] = 0$
3. $Q =$ set of vertices of graph G .
4. while Q is not empty

4.1 $u =$ vertex in Q with minimum distance.

4.2 remove u from Q .

4.3 for each neighbor v of u which is still in Q :

4.3.1 alt = $distance[u] + W(u,v)$

4.3.2 if alt < $distance[v]$

4.3.2.1 $distance[v] = alt$

4.3.2.2 $previous[v] = u$

5. return $distance[]$, $previous[]$.

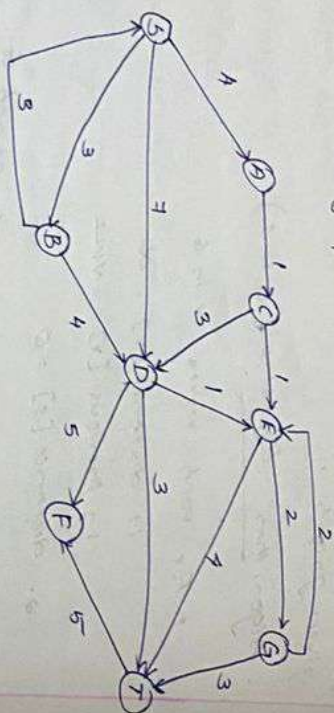
Complexity:-

- mainly depends on the implementation of Q .

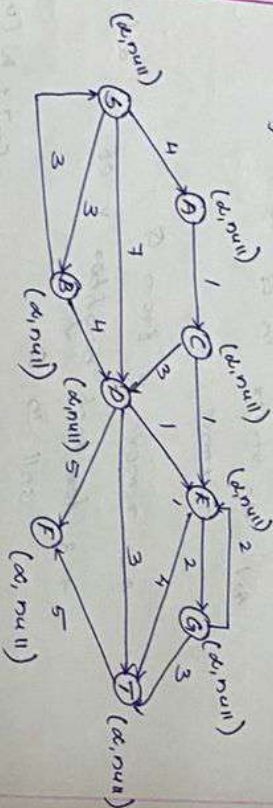
- mainly Q can be as array/linked list simple linear search.

- graph as adjacency list $O(E \log V)$.

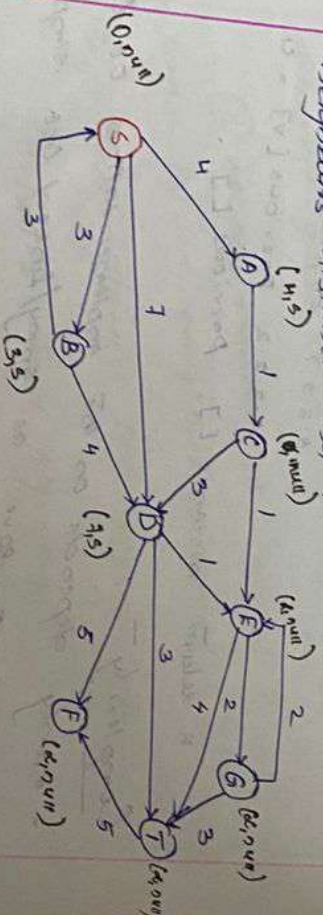
Example 1 -
Find the shortest path from S to all other vertices in the graph



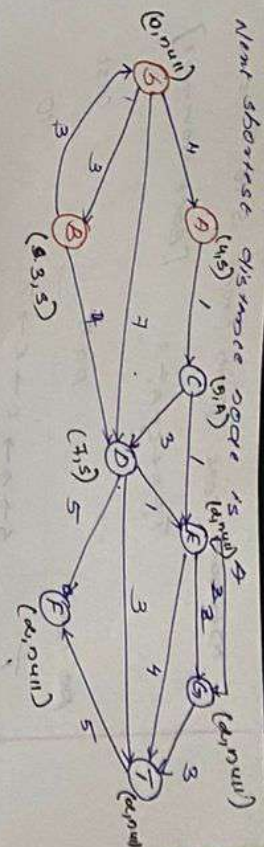
⇒ Initially set all distance as ∞ , and previous as null



• Set source node as S and from this update neighbours distance and previous.



Next shortest visited node is B [Neighbour of B is D then distance is to D from S is $4+3=7$ its already 7 so no updates]



Shortest: ∞ C $(4, D) \rightarrow E \rightarrow 5+1=6$ (6, C)

F, B, D, T

D $\rightarrow 5+3=8$ no update

* E (6, T)

← smallest

G $\rightarrow 6+2=8$ (8, E)
T $\rightarrow 6+4=10$ (10, E)

* D (7, F)

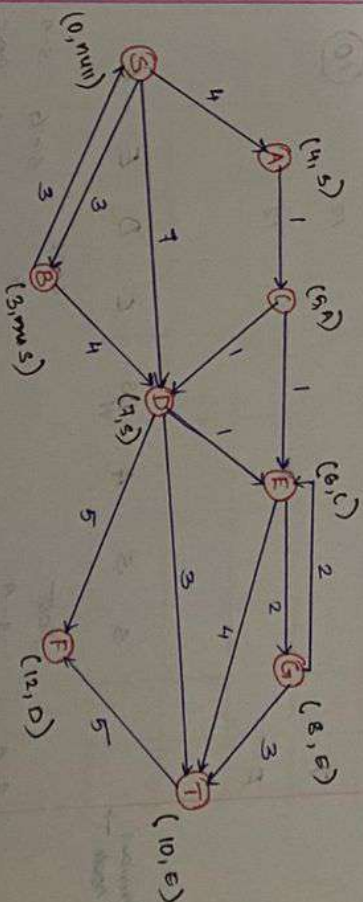
← smallest

T $\rightarrow 7+5=12$ (12, D)
F $\rightarrow 7+5=12$ (12, D)

* G (8, T)

* T (F)

8+3=11 No update
10+5=15 "



Cost from S to T = 10

Minimum cost path from S to T is:
 $T \leftarrow E \leftarrow C \leftarrow A \leftarrow S$ [Back traversal]

Path	Cost
S → A → C → E → T	10
S → D → F	12

From Node	S	A	B	C	D	E	F	G	T
S	0	∞	∞	∞	∞	∞	∞	∞	∞
B	4	(3)	∞	7	∞	∞	∞	∞	∞
A	(4)	∞	7	∞	∞	∞	∞	∞	∞
C		(5)	7	∞	∞	∞	∞	∞	∞
E			(7)	(6)	∞	∞	∞	∞	∞
D				(7)	∞	8	10	10	10
G						(8)			
T							(10)		
F						12			

Previous node →	S	A	B	C	D	E	F
S-A		3					
S-B			5				
S-C				7			
S-D					6		
S-E						12	
S-F							12